

INTRODUCTION TO SYNTAX I

Session 5: Clause structure II

Timea Szarvas

University of Potsdam

`timea.szarvas@uni-potsdam.de`

July 31st 2026

OVA 2026

TODAY'S PLAN

- Clause types
- Complementizers
- One full derivation

■ We landed on complementizers heading our clausal structure.

- ⊙ Complementizers precede the subject of the finite clause.
- ⊙ They form a constituent with the TP $\Rightarrow$ they are part of the embedded clause:

- (1) a. I wonder **whether** she will sleep.
b. **Whether** she will sleep, I wonder.
c. *She will sleep, I wonder **whether**.

- ⊙ But where is the complementizer located in our syntactic structure?

■ What is the function of the complementizer?

■ What is the function of the complementizer?

(2) a. I think [(that) David left].

b. *Hungarian*

Az-t hisz-em, [(hogy) Dávid el-men-t].

DEM-ACC believe-1SG COMP David PV-go-PST.3SG

'I think (that) David left.'

What is the function of the complementizer?

(2) a. I think [(that) David left].

b. *Hungarian*

Az-t hisz-em, [(hogy) Dávid el-men-t].

DEM-ACC believe-1SG COMP David PV-go-PST.3SG

'I think (that) David left.'

- ⊙ *David left* seems to be a complete TP, regardless of *that*.
- ⊙ The complementizer *that* is optional here but seems to **act as a bridge** between the main and the embedded clause.
- ⇒ The complementizer **does not belong to the TP**.

LOCATING THE COMPLEMENTIZER

(3) a. I [_{VP} **know** [_{?P} {
that
whether
if} David left]]

LOCATING THE COMPLEMENTIZER

- (3) a. I [_{VP} **know** [_{?P} {
that
whether
if
David left}]]
- b. I [_{VP} **wonder** [_{?P} {
*that
whether
if
David left}]]

LOCATING THE COMPLEMENTIZER

- (3) a. I [_{VP} **know** [_{?P} {
that
whether
if
} David left]]
- b. I [_{VP} **wonder** [_{?P} {
*that
whether
if
} David left]]
- c. I [_{VP} **think** [_{?P} {
that
*whether
*if
} David left]]

LOCATING THE COMPLEMENTIZER

- (3) a. I [_{VP} **know** [_{?P} {
that
whether
if } David left]]
- b. I [_{VP} **wonder** [_{?P} {
*that
whether
if } David left]]
- c. I [_{VP} **think** [_{?P} {
that
*whether
*if } David left]]

Verbs select complements, sometimes clausal ones.

- ⊙ The **complementizer** is indicative of the **clause type** (interrogative or declarative)
- ⇒ It depends on the complementizer whether the embedded clause can Merge with the main clause verb ⇒ the complementizer projects its features!

- **Clauses can be assigned θ -roles and take on different grammatical functions:**

■ Clauses can be assigned θ -roles and take on different grammatical functions:

- (4) a. You may know [(that) she fell asleep]. clause = complement of V $\Rightarrow$ object of *know*

■ Clauses can be assigned θ -roles and take on different grammatical functions:

- (4) a. You may know [(that) she fell asleep]. clause = complement of V $\Rightarrow$ object of *know*
b. [That she fell asleep] may surprise you. clause = specifier of T $\Rightarrow$ subject of *surprise*

■ Clauses can be assigned θ -roles and take on different grammatical functions:

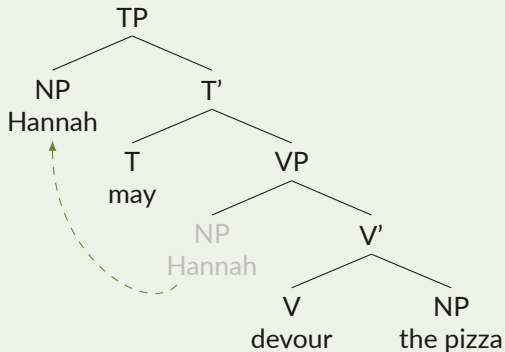
- (4) a. You may know [(that) she fell asleep]. clause = complement of V $\Rightarrow$ object of *know*
b. [That she fell asleep] may surprise you. clause = specifier of T $\Rightarrow$ subject of *surprise*
- ⊙ Notice that when the clause is the subject, the complementizer becomes obligatory ((4b) is bad if we omit *that*)!

THE TREE IS GROWING

I Our structure so far

THE TREE IS GROWING

Our structure so far

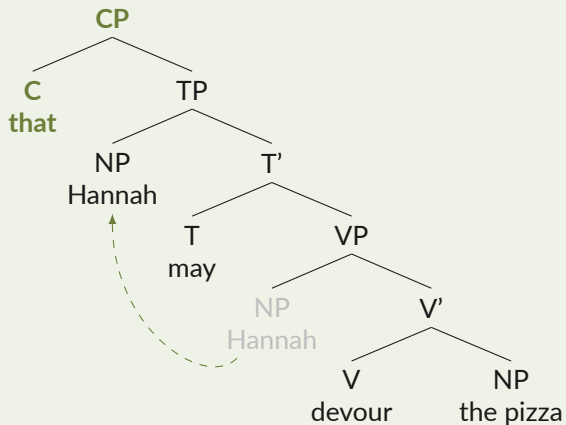


(5)

THE TREE IS GROWING

■ All signs point to the complementizer being a head.

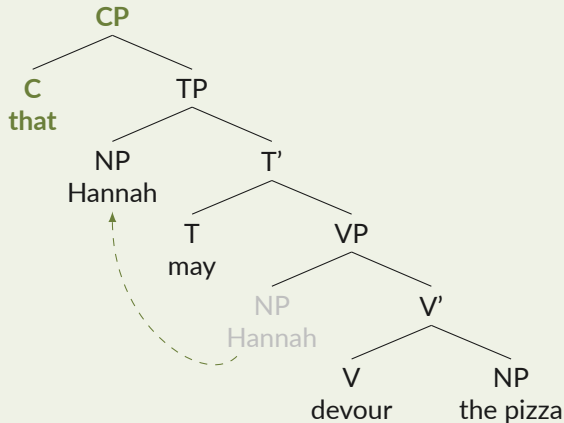
(6)



THE TREE IS GROWING

■ All signs point to the complementizer being a head.

(6)

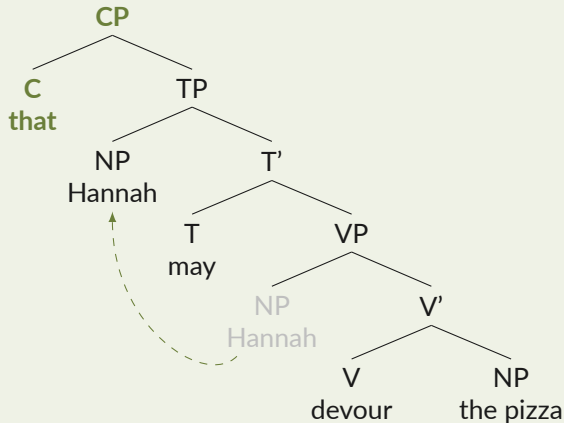


- ⊙ The complementizer **has the properties of a head** which (i) **selects a TP** as its complement and (ii) projects a phrase that can itself be **selected by a VP** either as a complement or specifier.

THE TREE IS GROWING

■ All signs point to the complementizer being a head.

(6)

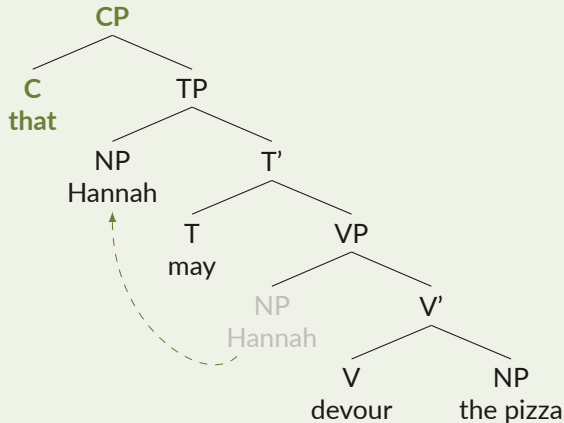


- ◉ The complementizer **has the properties of a head** which (i) **selects a TP** as its complement and (ii) projects a phrase that can itself be **selected by a VP** either as a complement or specifier.
- ◉ We label this **head C**(omplementizer).

THE TREE IS GROWING

■ All signs point to the complementizer being a head.

(6)



- ◉ The complementizer **has the properties of a head** which (i) **selects a TP** as its complement and (ii) projects a phrase that can itself be **selected by a VP** either as a complement or specifier.
- ◉ We label this **head C**(omplementizer).
- ◉ Clauses are of category [C], verbs that select clauses have the c-selectional feature [uC].

(Apologies for absolutely everything being called a (distinct!) c-something, the field could have done better.)

- CP is the topmost projection of the clause.

I CP is the topmost projection of the clause.

- ⊙ We can now formally model the observation that **language is recursive** by embedding CPs into one another:

I CP is the topmost projection of the clause.

- ⊙ We can now formally model the observation that **language is recursive** by embedding CPs into one another:

[_{TP} I [_{VP} think

]]

■ CP is the topmost projection of the clause.

- ⊙ We can now formally model the observation that **language is recursive** by embedding CPs into one another:

[_{TP} I [_{VP} think
[_{CP} that [_{TP} the mouse [_{VP} knows

]]]

]]

■ CP is the topmost projection of the clause.

- ⊙ We can now formally model the observation that **language is recursive** by embedding CPs into one another:

[_{TP} I [_{VP} think
 [_{CP} that [_{TP} the mouse [_{VP} knows
 [_{CP} that [_{TP} the cat [_{VP} saw

]]]

]]]

]]

■ CP is the topmost projection of the clause.

- ⊙ We can now formally model the observation that **language is recursive** by embedding CPs into one another:

[_{TP} I [_{VP} think
 [_{CP} that [_{TP} the mouse [_{VP} knows
 [_{CP} that [_{TP} the cat [_{VP} saw
 [_{CP} that [_{TP} the dog [_{VP} slept.
]]]
]]]
]]]
]]

■ Let's look at some cross-linguistic patterns!

- ⊙ In German, spelling out the C head has an impact on word order when an auxiliary is present:

■ Let's look at some cross-linguistic patterns!

- ⊙ In German, spelling out the C head has an impact on word order when an auxiliary is present:

(7) *German*

- a. Finn hat geschlafen.
Finn have.3SG slept
'Finn slept.'

■ Let's look at some cross-linguistic patterns!

- ⊙ In German, spelling out the C head has an impact on word order when an auxiliary is present:

(7) *German*

- a. Finn hat geschlafen.
Finn have.3SG slept
'Finn slept.'
- b. Sarah denk-t, Finn **hat** **geschlafen**.
Sarah think-3SG Finn have.3SG slept
'Sarah thinks Finn slept.'

■ Let's look at some cross-linguistic patterns!

- In German, spelling out the C head has an impact on word order when an auxiliary is present:

(7) *German*

- Finn *hat* geschlafen.
Finn have.3SG slept
'Finn slept.'
- Sarah *denk-t*, Finn *hat* geschlafen.
Sarah think-3SG Finn have.3SG slept
'Sarah thinks Finn slept.'
- Sarah *denk-t*, *dass* Finn *geschlafen hat*.
Sarah think-3SG COMP Finn slept have.3SG
'Sarah thinks that Finn slept.'

■ Complementizers are obligatory in Scottish Gaelic and French:

(8) *Scottish Gaelic*

- a. Thuairt mi **gu** bheil i tinn.
say-PST I COMP was she ill
'I said that she was ill.'
- b. *Thuairt mi bheil i tinn.
say-PST I was she ill

(9) *French*

- a. J'ai dit **qu'** elle était malade
I-have said COMP she was ill
'I said that she was ill.'
- b. *J'ai dit elle était malade
I-have said she was ill

ALMOST QUESTIONS

- (10) a. I know that **Hannah** might devour the pizza.
b. I know (*that) **who** might devour the pizza.
c. I know (*that) **which pizza** Hannah will devour.

- (10) a. I know that **Hannah** might devour the pizza.
b. I know (*that) **who** might devour the pizza.
c. I know (*that) **which pizza** Hannah will devour.

■ Describe these pairs. What do we observe?

- (10) a. I know that **Hannah** might devour the pizza.
b. I know (*that) **who** might devour the pizza.
c. I know (*that) **which pizza** Hannah will devour.

Describe these pairs. What do we observe?

- English has SVO word order, with a subject in SpecTP and the object as the complement of the verb.

- (10) a. I know that **Hannah** might devour the pizza.
b. I know (*that) **who** might devour the pizza.
c. I know (*that) **which pizza** Hannah will devour.

Describe these pairs. What do we observe?

- English has SVO word order, with a subject in SpecTP and the object as the complement of the verb.
- But why does the **object precede the embedded subject** in (10d)?
- And where did the **complementizer** go in (10b) and (10d)?

■ *Who and which pizza differ from Hannah and the pizza.*

- ⊙ They are still the arguments of the predicate *devour* in (10), omitting them is bad:

- (11) a. *I know might devour the pizza.
b. *I know Hannah will devour.

■ *Who and which pizza differ from Hannah and the pizza.*

- ⊙ They are still the arguments of the predicate *devour* in (10), omitting them is bad:

- (11) a. *I know might devour the pizza.
b. *I know Hannah will devour.

- ⊙ But they are sensitive to the C head because they cannot co-occur with *that*.

■ *Who* and *which pizza* differ from *Hannah* and *the pizza*.

- ◉ They are still the arguments of the predicate *devour* in (10), omitting them is bad:

- (11) a. *I know might devour the pizza.
b. *I know Hannah will devour.

- ◉ But they are sensitive to the C head because they cannot co-occur with *that*.

■ We have previously argued based on complementary distribution.

- ◉ We declared modals, emphatic *do* and infinitival *to* T heads because they could not co-occur (recall session 4).
- ◉ Given that *who* and *which pizza* seem to be in complementary distribution with the C head *that*, should we assume that they are C heads, too?

■ **Absolutely not! Several of their properties are at odds with being C heads:**

■ Absolutely not! Several of their properties are at odds with being C heads:

- ⊙ They are of the lexical category [N] denoting real life entities (whose precise identity we are questioning).

■ Absolutely not! Several of their properties are at odds with being C heads:

- ⊙ They are of the lexical category [N] denoting real life entities (whose precise identity we are questioning).
- ⊙ They receive θ -roles from predicates and act as their arguments.

■ Absolutely not! Several of their properties are at odds with being C heads:

- ⊙ They are of the lexical category [N] denoting real life entities (whose precise identity we are questioning).
- ⊙ They receive θ -roles from predicates and act as their arguments.

■ How do verbs select their arguments and assign θ -roles to them?

■ Absolutely not! Several of their properties are at odds with being C heads:

- ⊙ They are of the lexical category [N] denoting real life entities (whose precise identity we are questioning).
- ⊙ They receive θ -roles from predicates and act as their arguments.

■ How do verbs select their arguments and assign θ -roles to them?

- ⊙ Under sisterhood $\Rightarrow$ locality (of c-command)!

■ Absolutely not! Several of their properties are at odds with being C heads:

- ⊙ They are of the lexical category [N] denoting real life entities (whose precise identity we are questioning).
- ⊙ They receive θ -roles from predicates and act as their arguments.

■ How do verbs select their arguments and assign θ -roles to them?

- ⊙ Under sisterhood $\Rightarrow$ locality (of c-command)!
- ⊙ So how can *who* and *which pizza* be locally selected by *devour* and simultaneously be in a position that interacts with the C head?

■ Absolutely not! Several of their properties are at odds with being C heads:

- ⊙ They are of the lexical category [N] denoting real life entities (whose precise identity we are questioning).
- ⊙ They receive θ -roles from predicates and act as their arguments.

■ How do verbs select their arguments and assign θ -roles to them?

- ⊙ Under sisterhood $\Rightarrow$ locality (of c-command)!
 - ⊙ So how can *who* and *which pizza* be locally selected by *devour* and simultaneously be in a position that interacts with the C head?
- $\Rightarrow$ **In other words, how can something behave like it's in two positions at once?**

■ Recall the EPP feature on the T head.

- ◉ We saw that English finite clauses require a subject (structurally higher than auxiliaries).
- ⇒ To account for this, we assumed that the T head has a **requirement to fill its specifier**.
- ◉ We modeled this requirement by postulating the EPP-feature, either attracting the noun/clause from SpecVP or requiring an expletive if the verb selects no arguments.

¹We call them that because most of them start with wh- in English (*who, what, which, where, when, why*, and then there's *how*.)

■ Recall the EPP feature on the T head.

- ⊙ We saw that English finite clauses require a subject (structurally higher than auxiliaries).
- ⇒ To account for this, we assumed that the T head has a **requirement to fill its specifier**.
- ⊙ We modeled this requirement by postulating the EPP-feature, either attracting the noun/clause from SpecVP or requiring an expletive if the verb selects no arguments.

■ We will assume that the C head has a similar requirement.

- ⊙ C, being a functional head like T, also wants its specifier to be filled.

¹We call them that because most of them start with wh- in English (*who, what, which, where, when, why*, and then there's *how*.)

■ Recall the EPP feature on the T head.

- ⊙ We saw that English finite clauses require a subject (structurally higher than auxiliaries).
- ⇒ To account for this, we assumed that the T head has a **requirement to fill its specifier**.
- ⊙ We modeled this requirement by postulating the EPP-feature, either attracting the noun/clause from SpecVP or requiring an expletive if the verb selects no arguments.

■ We will assume that the C head has a similar requirement.

- ⊙ C, being a functional head like T, also wants its specifier to be filled.
- ⊙ C's purpose is marking clause type: it is sensitive to, among others, *wh-phrases*.¹

¹We call them that because most of them start with *wh-* in English (*who, what, which, where, when, why*, and then there's *how*.)

■ The feature on C can be satisfied via Merge.

- ⊙ In embedded declaratives, a complementizer is Merged in the head position (*that*).
- ⊙ In embedded interrogatives, an interrogative complementizer is Merged in the head position (*whether/if*).

FORMALIZING C'S REQUIREMENTS

■ The feature on C can be satisfied via Merge.

- ◉ In embedded declaratives, a complementizer is Merged in the head position (*that*).
- ◉ In embedded interrogatives, an interrogative complementizer is Merged in the head position (*whether/if*).

■ The feature on C can also be satisfied via movement.

- ◉ If there is a suitable phrase in the structure, C attracts it to its specifier.
- ⇒ C attracts constituents in its c-command domain that have a [Q] feature (relevant for clause type, information structure, etc.)
- ◉ We have only looked at wh-phrases moving to SpecCP, but C attracts several types of phrases that carry relevant for clause type!

APPLYING EVERYTHING WE HAVE LEARNED SO FAR

■ Let's derive *I know which pizza Hannah may devour*.

APPLYING EVERYTHING WE HAVE LEARNED SO FAR

(12)

V	NP
devour	which pizza
[uN, uN]	[+Q]

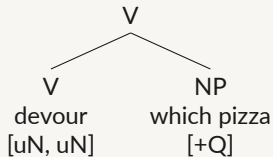
APPLYING EVERYTHING WE HAVE LEARNED SO FAR

(12)

V
devour
[uN, uN]

NP
which pizza
[+Q]

(13)



APPLYING EVERYTHING WE HAVE LEARNED SO FAR

(12)

V NP
devour which pizza
[uN, uN] [+Q]

(13)

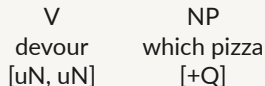
 V
 / \
 V NP
 devour which pizza
 [uN, uN] [+Q]

(14)

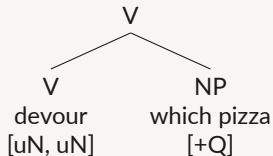
 V' [uN]
 / \
 V NP
 devour which pizza
 [uN, uN] [+Q]

APPLYING EVERYTHING WE HAVE LEARNED SO FAR

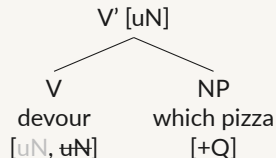
(12)



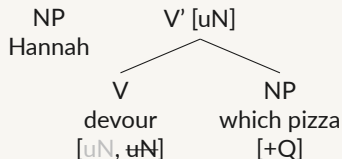
(13)



(14)

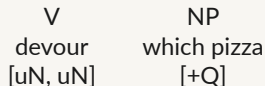


(15)

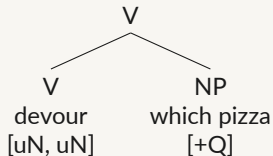


APPLYING EVERYTHING WE HAVE LEARNED SO FAR

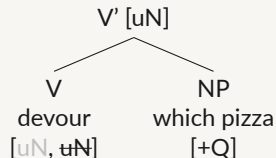
(12)



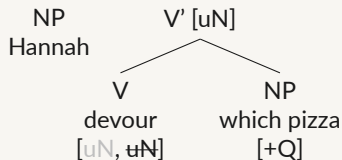
(13)



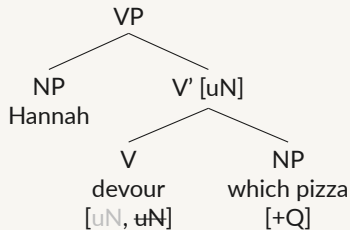
(14)



(15)

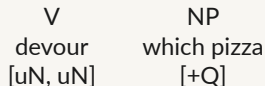


(16)

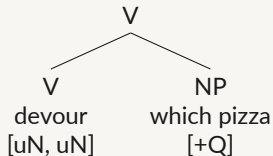


APPLYING EVERYTHING WE HAVE LEARNED SO FAR

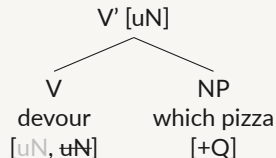
(12)



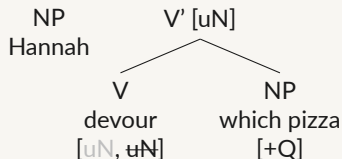
(13)



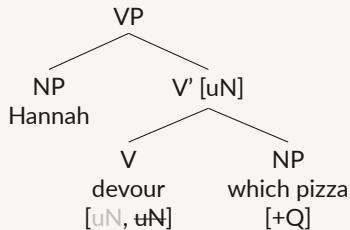
(14)



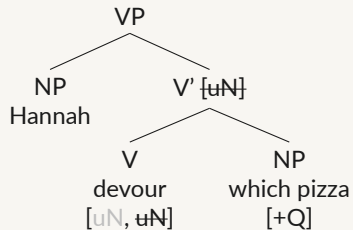
(15)



(16)

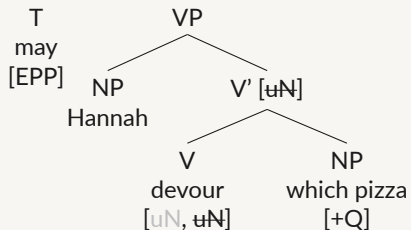


(17)



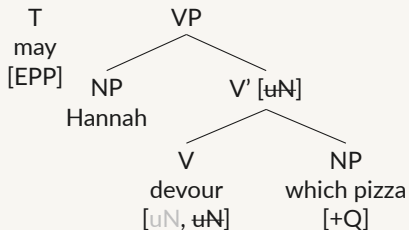
APPLYING EVERYTHING WE HAVE LEARNED SO FAR

(18)

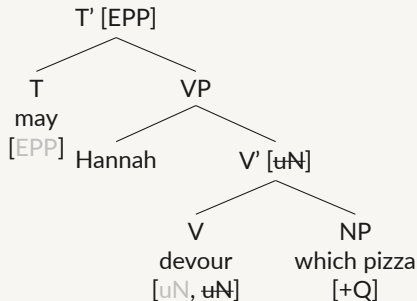


APPLYING EVERYTHING WE HAVE LEARNED SO FAR

(18)

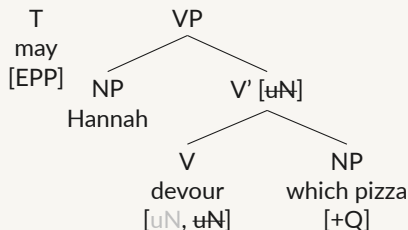


(19)

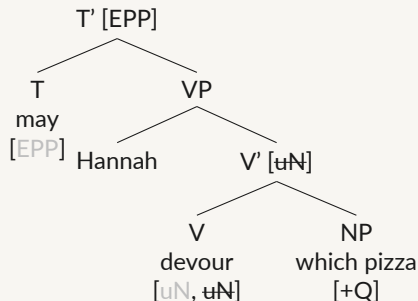


APPLYING EVERYTHING WE HAVE LEARNED SO FAR

(18)



(19)

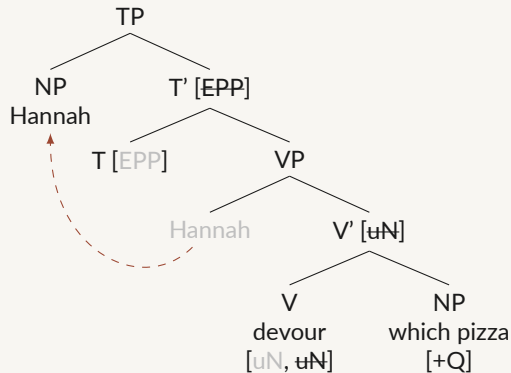


■ **Reminder: functional heads do not have an argument structure like lexical heads do.**

- ⊙ While elements within the lexical projection combine based on the respective predicates' argument structure, functional heads like T and C have more rigid requirements to build the clausal spine: T needs a VP, C needs a TP.

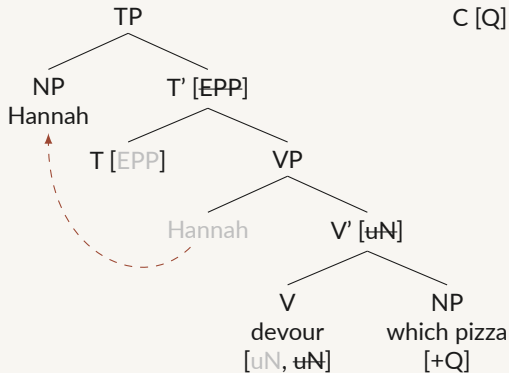
APPLYING EVERYTHING WE HAVE LEARNED SO FAR

(20)

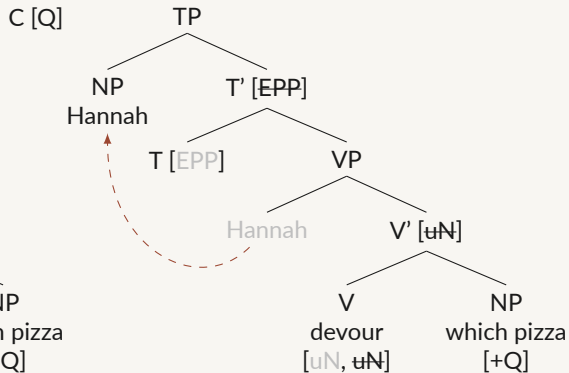


APPLYING EVERYTHING WE HAVE LEARNED SO FAR

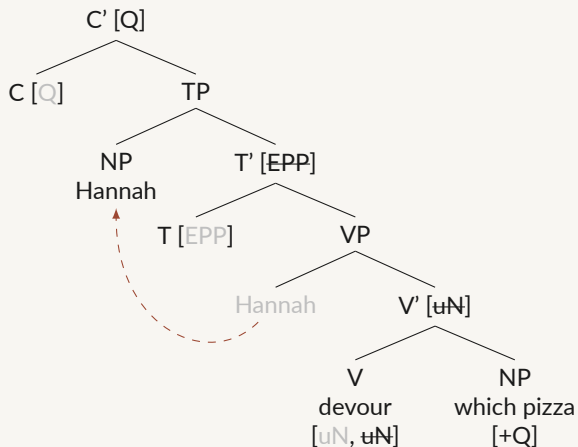
(20)



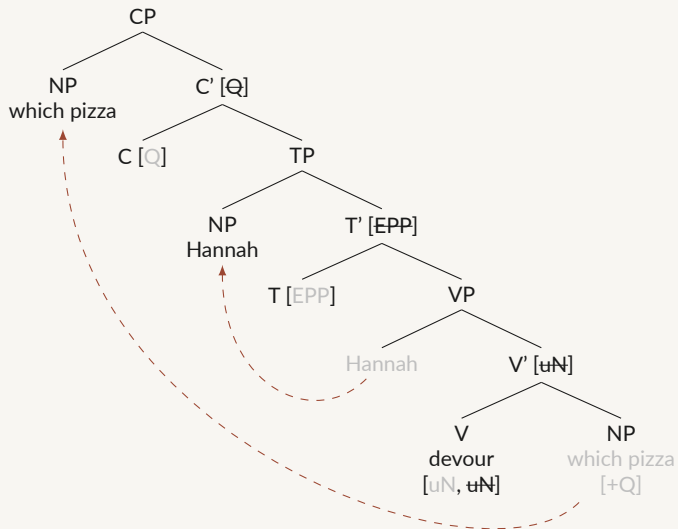
(21)



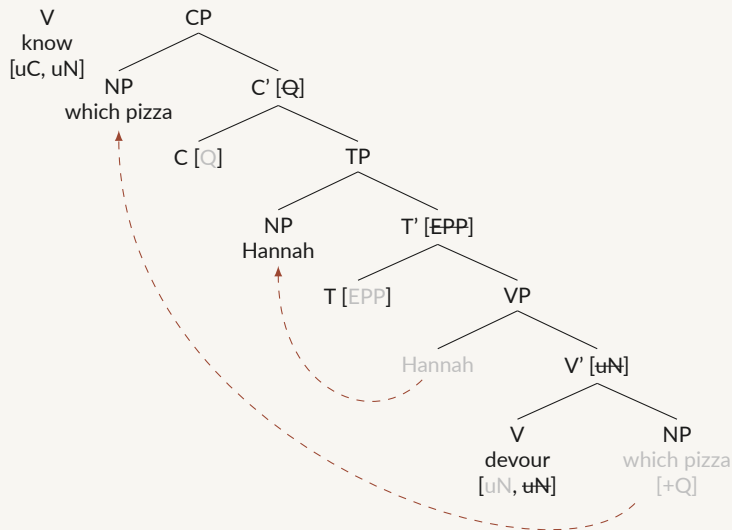
APPLYING EVERYTHING WE HAVE LEARNED SO FAR



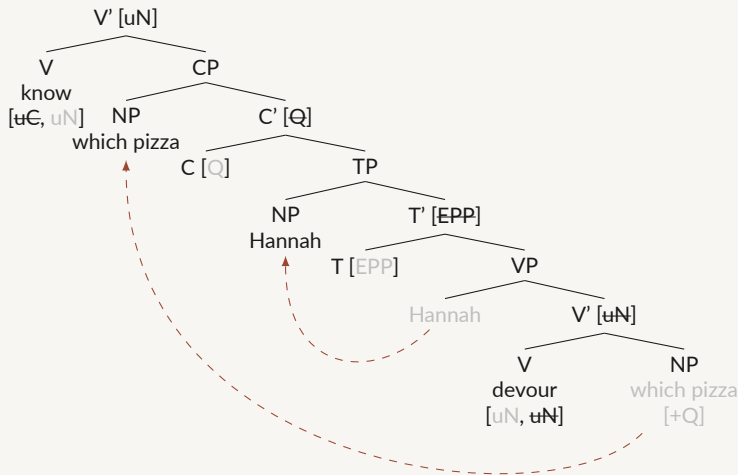
APPLYING EVERYTHING WE HAVE LEARNED SO FAR



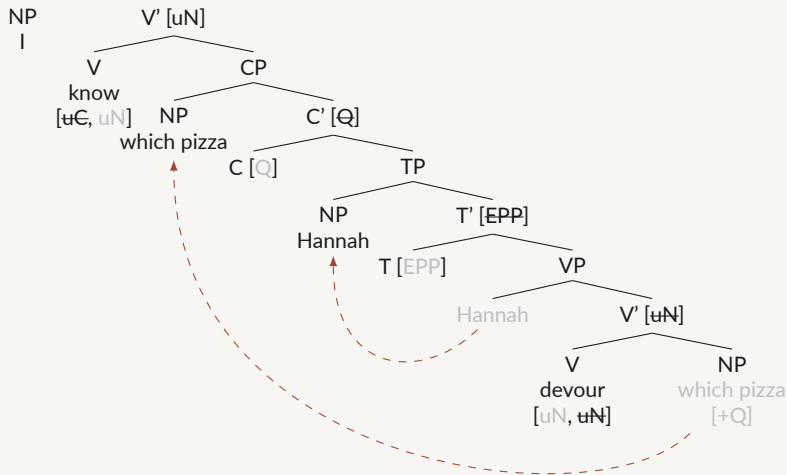
APPLYING EVERYTHING WE HAVE LEARNED SO FAR



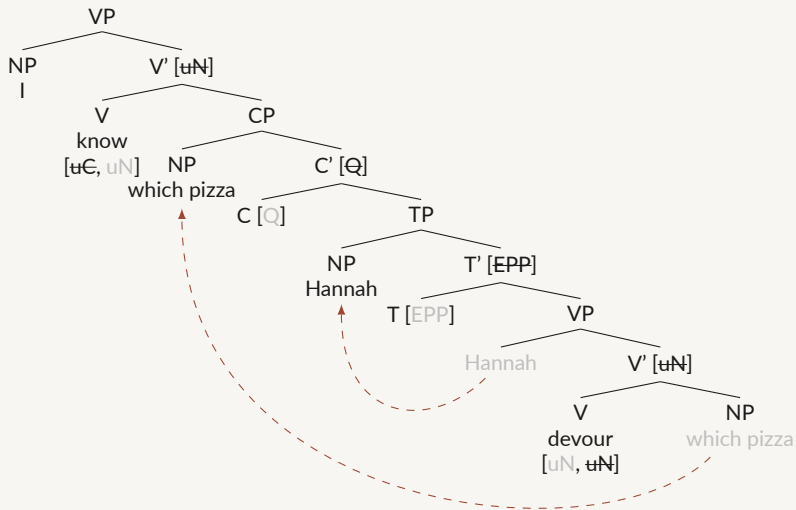
APPLYING EVERYTHING WE HAVE LEARNED SO FAR



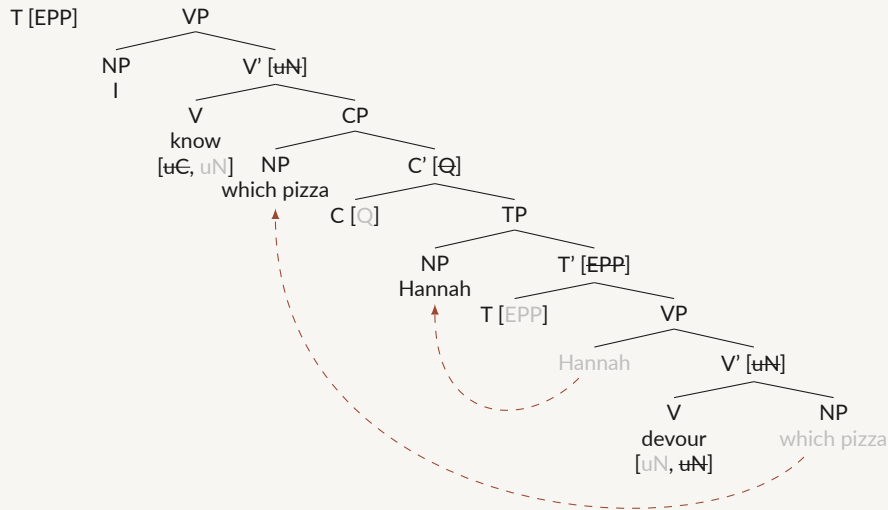
APPLYING EVERYTHING WE HAVE LEARNED SO FAR



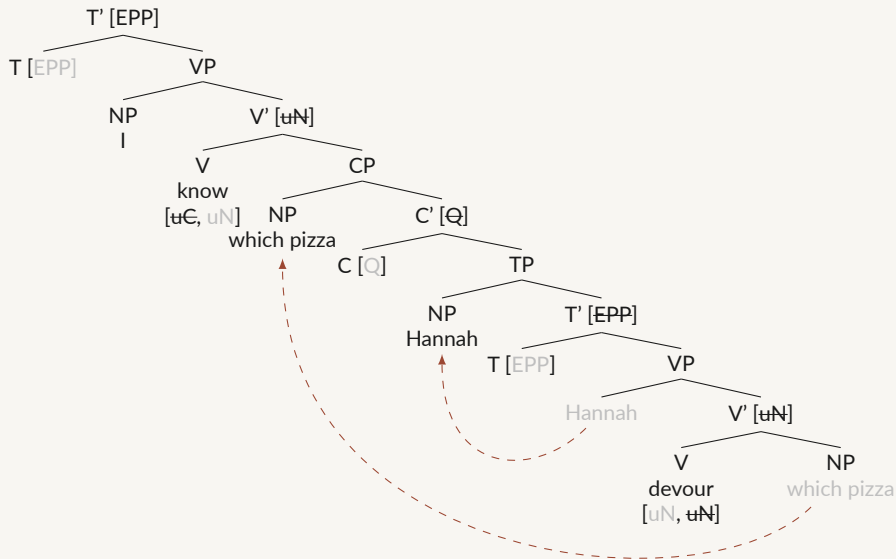
APPLYING EVERYTHING WE HAVE LEARNED SO FAR



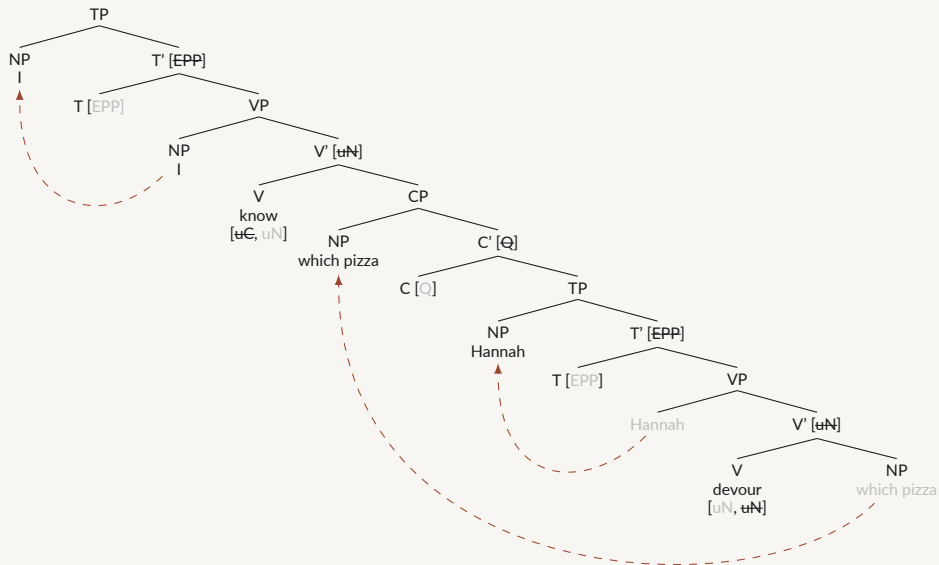
APPLYING EVERYTHING WE HAVE LEARNED SO FAR



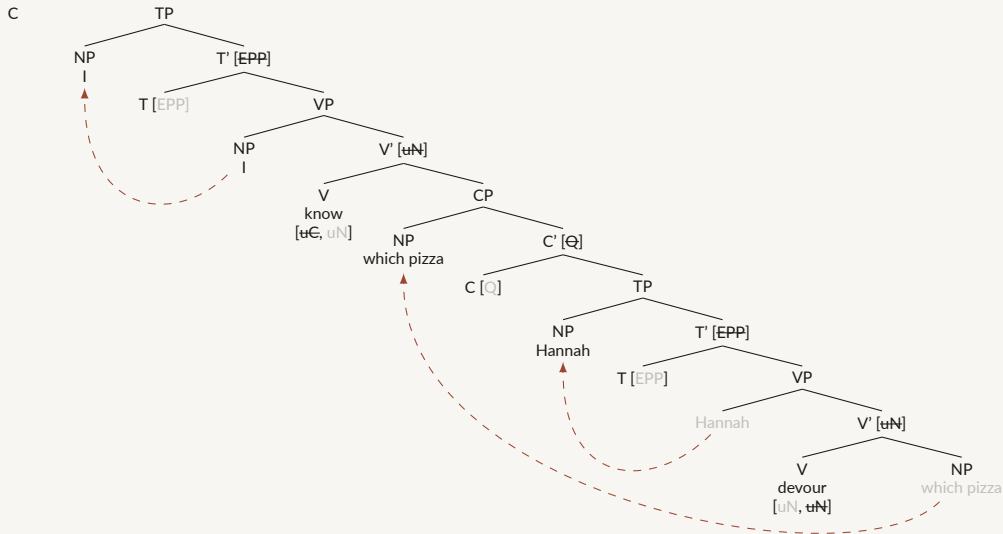
APPLYING EVERYTHING WE HAVE LEARNED SO FAR



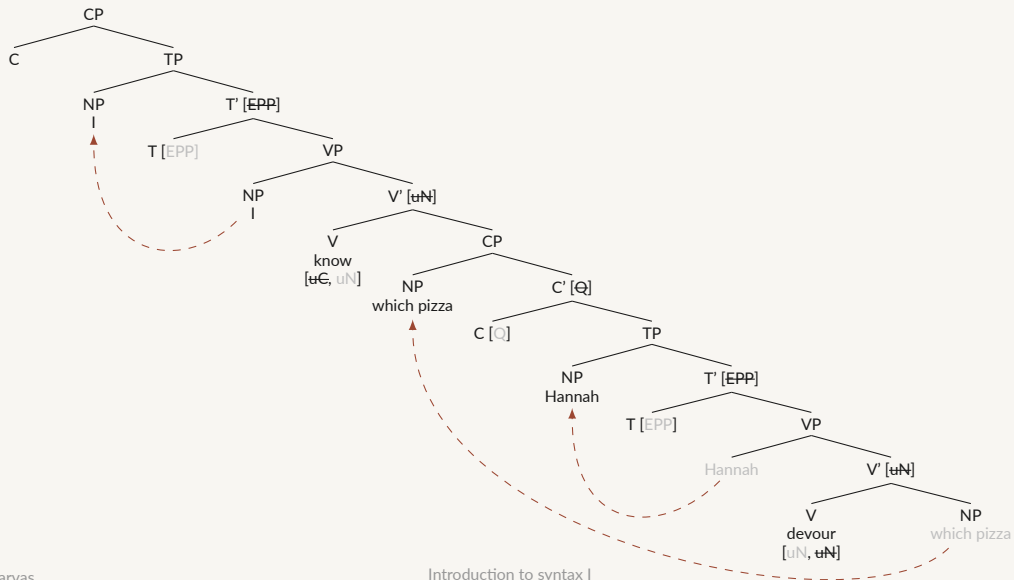
APPLYING EVERYTHING WE HAVE LEARNED SO FAR



APPLYING EVERYTHING WE HAVE LEARNED SO FAR



APPLYING EVERYTHING WE HAVE LEARNED SO FAR



I Recall the term 'base generation'

- ⊙ We briefly mentioned in session 4 that the subject is **base generated** (i.e. Merged) in SpecVP but ends up **moving** to SpecTP in our model.

■ Recall the term 'base generation'

- ⊙ We briefly mentioned in session 4 that the subject is **base generated** (i.e. Merged) in SpecVP but ends up **moving** to SpecTP in our model.
- ⊙ We now saw wh-phrases that are base generated in the lexical projection (VP), too, and end up moving to SpecCP.

I Recall the term 'base generation'

- ◉ We briefly mentioned in session 4 that the subject is **base generated** (i.e. Merged) in SpecVP but ends up **moving** to SpecTP in our model.
- ◉ We now saw wh-phrases that are base generated in the lexical projection (VP), too, and end up moving to SpecCP.
- ◉ When referring to a **constituent's origin**, i.e. the position where it was base generated/Merged, we often use the terms **base position** or **underlying position**.

UNDERLYING VS. SURFACE POSITIONS

I Recall the term 'base generation'

- ◉ We briefly mentioned in session 4 that the subject is **base generated** (i.e. Merged) in SpecVP but ends up **moving** to SpecTP in our model.
- ◉ We now saw wh-phrases that are base generated in the lexical projection (VP), too, and end up moving to SpecCP.
- ◉ When referring to a **constituent's origin**, i.e. the position where it was base generated/Merged, we often use the terms **base position** or **underlying position**.
- ◉ When referring to a constituent's final destination, i.e. the position to which it moves, we often use the terms **(final) landing site**, **derived position** or **surface position**.

UNDERLYING VS. SURFACE POSITIONS

■ Representing underlying vs. surface positions

- ⊙ Every constituent has a base position; the ones that move also have a landing site.
- ⊙ In earlier theories (Government & Binding, $\bar{x}$ -theory), sentences were modeled to have two levels: D-structure (for *deep*) and S-structure (for *surface*).

I Representing underlying vs. surface positions

- ⊙ Every constituent has a base position; the ones that move also have a landing site.
- ⊙ In earlier theories (Government & Binding, $\bar{x}$ -theory), sentences were modeled to have two levels: D-structure (for *deep*) and S-structure (for *surface*).
- ⊙ In Minimalism, syntax is thought of as a **computational system triggering processes that terminate in a final output** rather than levels of representation.

UNDERLYING VS. SURFACE POSITIONS

I Representing underlying vs. surface positions

- ◉ Every constituent has a base position; the ones that move also have a landing site.
- ◉ In earlier theories (Government & Binding, $\bar{x}$ -theory), sentences were modeled to have two levels: D-structure (for *deep*) and S-structure (for *surface*).
- ◉ In Minimalism, syntax is thought of as a **computational system triggering processes that terminate in a final output** rather than levels of representation.

I Syntax keeps track of every step of the computational process.

- ◉ Recall structural ambiguities (session 1): silent syntactic structure reveals itself via the consequences it has for the interpretation (LF) and pronunciation (PF)!
- ◉ **Prediction:** if syntactic structures are built in a computational process, there should be evidence (on PF and/or LF) for each step.

SUMMARY OF TODAY

- C is another functional head in the clausal spine.

SUMMARY OF TODAY

■ C is another functional head in the clausal spine.

- ⊙ C expresses **clause type**; T, we saw yesterday, expresses **tense and inflection**.

SUMMARY OF TODAY

■ C is another functional head in the clausal spine.

- ⊙ C expresses **clause type**; T, we saw yesterday, expresses **tense and inflection**.
- ⊙ Languages vary with respect to how they express the C head: the complementizer is optional in some and obligatory in other languages.

SUMMARY OF TODAY

■ C is another functional head in the clausal spine.

- ⊙ C expresses **clause type**; T, we saw yesterday, expresses **tense and inflection**.
- ⊙ Languages vary with respect to how they express the C head: the complementizer is optional in some and obligatory in other languages.

■ Syntactic structure can be silent; one constituent can satisfy the requirements of two heads

- ⇒ The **lexical head** selecting them in their base positions via Merge under local c-command (c-selectional feature checking under sisterhood).
- ⇒ The **functional head** that attracts them to their surface position under non-local c-command (filling the specifier position of T/C).

SUMMARY OF TODAY

■ C is another functional head in the clausal spine.

- ⊙ C expresses **clause type**; T, we saw yesterday, expresses **tense and inflection**.
- ⊙ Languages vary with respect to how they express the C head: the complementizer is optional in some and obligatory in other languages.

■ Syntactic structure can be silent; one constituent can satisfy the requirements of two heads

- ⇒ The **lexical head** selecting them in their base positions via Merge under local c-command (c-selectional feature checking under sisterhood).
- ⇒ The **functional head** that attracts them to their surface position under non-local c-command (filling the specifier position of T/C).
- ⊙ The default rule for the languages you have seen so far is to take the final position into account for pronunciation (i.e. PF references only the final landing site).
- ⊙ Semantically, base positions matter a lot (LF often references base positions).

Thank you for attending!

Questions?